

102. Binary Tree Level Order Traversal Medium

Acceptance: 72.0% | [LeetCode](#)

Given the `root` of a binary tree, return *the level order traversal of its nodes' values*. (i.e., from left to right, level by level).

Example 1:

```
**Input:** root = [3,9,20,null,null,15,7]
**Output:** [[3],[9,20],[15,7]]
```

Example 2:

```
**Input:** root = [1]
**Output:** [[1]]
```

Example 3:

```
**Input:** root = []
**Output:** []
```

Constraints:

- The number of nodes in the tree is in the range `[0, 2000]`.
- `-1000 <= Node.val <= 1000`

Claude Opus Solution

Binary Tree Level Order Traversal

1. Intuition

The key insight is that **"level by level" traversal naturally maps to Breadth-First Search (BFS)**.

- DFS goes *deep* (down one path before backtracking) — not ideal here.
- BFS goes *wide* (visits all nodes at the current depth before moving deeper) — perfect for grouping nodes by level.

The crucial trick: **before processing each level, record the current queue size**. That size tells us exactly how many nodes belong to the current level. We process precisely that many nodes, collecting their values into one sublist, while enqueueing their children for the next level.

This "snapshot the queue length" technique is what cleanly separates levels.

2. Approach (Pseudocode)

```
if root is None: return []

queue = [root]          # FIFO queue, initialized with root
result = []

while queue is not empty:
    level_size = length(queue)  # number of nodes in current level
    level = []
    repeat level_size times:
        node = dequeue from front
        add node.val to level
        if node.left: enqueue node.left
        if node.right: enqueue node.right
    add level to result

return result
```

3. Python Solution

```
import collections
from typing import Optional, List

# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```

```

class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        # Edge case: empty tree -> return empty list
        if root is None:
            return []

        # deque gives O(1) pops from the left (FIFO behavior for BFS).
        queue = collections.deque([root])
        output_list = []

        # Process the tree one level at a time until queue is empty.
        while queue:
            level = [] # values collected for the current level

            # KEY STEP: snapshot the current number of nodes.
            # All these nodes belong to the SAME level. Their children
            # (added during this loop) belong to the NEXT level, but since
            # we only loop `len(queue)` times, we won't touch them now.
            for _ in range(len(queue)):
                node = queue.popleft() # take next node in current level
                level.append(node.val) # record its value

            # Enqueue children -> they form the next level.
            if node.left is not None:
                queue.append(node.left)
            if node.right is not None:
                queue.append(node.right)

            # Finished one full level; store it.
            output_list.append(level)

        return output_list

```

4. Complexity Analysis

Time Complexity: $O(n)$ Each of the n nodes is enqueued once and dequeued once. Each operation (append, popleft, value read) is $O(1)$, so total work is linear.

Space Complexity: $O(n)$ - The **queue** holds at most one level of nodes at a time. In the worst case (a complete binary tree), the last level contains up to $n/2$ nodes $\rightarrow O(n)$. - The **output list** stores all n values $\rightarrow O(n)$.

So overall **$O(n)$** auxiliary space.

5. Alternative Approaches

A) DFS with depth tracking (Recursive)

Pass the current depth down the recursion. Use the depth as an index into the result list — if `depth == len(result)`, start a new level.

```

class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        result = []

        def dfs(node, depth):
            if node is None:
                return
            # First time reaching this depth -> create a new sublist.
            if depth == len(result):
                result.append([])
            result[depth].append(node.val) # append to correct level
            dfs(node.left, depth + 1)
            dfs(node.right, depth + 1)

        dfs(root, 0)
        return result

```

- **Time:** $O(n)$, **Space:** $O(h)$ recursion stack (h = tree height; $O(n)$ worst case for skewed trees).

- *Caveat:* Values are appended left-to-right correctly because we always recurse left before right.

B) BFS with two lists instead of length-snapshot

Maintain `current_level` and build `next_level`, then swap. Functionally identical to the size-snapshot trick but uses slightly more explicit bookkeeping.

Pattern Summary

This is the canonical "**Level Order / BFS template**" problem. The same length-snapshot pattern solves many variants: - **Zigzag traversal** (reverse alternate levels) - **Right-side view** (take last node of each level) - **Level averages / max per level** - **Bottom-up level order** (just reverse the final result)

Master this template and a whole family of tree problems becomes trivial.

Codex Solution

1. Intuition

Level order traversal means visiting nodes **level by level**, from **left to right**.

This is exactly the pattern of **Breadth-First Search, BFS**.

For BFS, we use a **queue**:

- Start by putting the root node into the queue.
- While the queue is not empty:
- Process all nodes currently in the queue.
- These nodes belong to the same level.
- Add their children to the queue for the next level.

The key trick is to record the queue size at the start of each level:

```
level_size = len(queue)
```

This tells us how many nodes belong to the current level.

2. Approach

Pseudocode

```
function levelOrder(root):
    if root is null:
        return []

    result = []
    queue = new queue containing root

    while queue is not empty:
        level = []
        level_size = size of queue

        repeat level_size times:
            node = queue.pop_front()
            add node.val to level

            if node.left exists:
                queue.push_back(node.left)

            if node.right exists:
                queue.push_back(node.right)

        add level to result

    return result
```

3. Python Solution

```
from collections import deque
from typing import Optional, List

# Definition for a binary tree node.
# LeetCode provides this class, so usually you do not need to redefine it.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        # If the tree is empty, there are no levels to return.
        if root is None:
            return []

        # This will store the final level order traversal.
        result = []

        # Queue is used for BFS.
        # We start BFS from the root node.
        queue = deque([root])

        # Continue while there are still nodes to process.
        while queue:
            # This list stores all node values in the current level.
            current_level = []

            # The number of nodes currently in the queue
            # is exactly the number of nodes in this level.
            level_size = len(queue)

            # Process only the nodes from the current level.
            for _ in range(level_size):
                # Remove the front node from the queue.
                node = queue.popleft()

                # Add its value to the current level.
                current_level.append(node.val)

                # Add the left child to the queue if it exists.
                # It will be processed in the next level.
                if node.left is not None:
                    queue.append(node.left)

                # Add the right child to the queue if it exists.
                # It will also be processed in the next level.
                if node.right is not None:
                    queue.append(node.right)

            # After processing all nodes in this level,
            # add the level list to the final result.
            result.append(current_level)

        return result
```

4. Complexity Analysis

Let n be the number of nodes in the binary tree.

Time Complexity: $O(n)$

Each node is visited exactly once.

For every node, we:

- Remove it from the queue once.

- Add its value to a list once.
- Check its left and right children once.

Therefore, the total time complexity is:

$O(n)$

Space Complexity: $O(n)$

The queue can contain many nodes at once.

In the worst case, the last level of a binary tree can contain up to about $n / 2$ nodes.

The result list also stores all node values.

Therefore, the total space complexity is:

$O(n)$

5. Alternative Approaches

DFS by Depth

We can also solve this using **Depth-First Search**.

The idea is:

- Traverse the tree recursively.
- Keep track of the current depth.
- If this is the first time visiting this depth, create a new list.
- Add the current node value to the list for that depth.

Example pattern:

```
def dfs(node, depth):
    if node is None:
        return

    if depth == len(result):
        result.append([])

    result[depth].append(node.val)

    dfs(node.left, depth + 1)
    dfs(node.right, depth + 1)
```

DFS also has:

- Time complexity: $O(n)$
- Space complexity: $O(n)$

However, BFS is usually the most natural approach for level order traversal.